

FÍSICA PARA JOGOS DIGITAIS · GRAU B

Benchmark de Physics Engines

Colocando a arquitetura à prova — estabilidade & estresse de colisão

Unity (PhysX) · **Unreal** (Chaos) · **Godot** (Jolt)

Docente: Prof.^a Rossana Baptista Queiroz

Acadêmico: Matheus Dubin da Silveira

Universidade do Vale do Rio dos Sinos · Escola Politécnica · 2026

01

Introdução

Da teoria à prova

O **Grau A** levantou as arquiteturas internas (solvers, threading, CCD). O **Grau B** coloca isso à prova: medir os limites e gargalos reais via testes de estresse, com a mesma cena nas três engines e rigor estatístico.

- **Cenário 1 — A Torre:** pilha alta de corpos rígidos. Mede **estabilidade**: tempo até repouso (sleep), jitter e colapso.
- **Cenário 2 — A Chuva:** 1k / 5k / 10k corpos caindo em caixa fechada. Mede **desempenho**: Physics Step Time e FPS.
- **Regra:** malhas primitivas, mesma massa/atrito, **fixed timestep 0,02 s (50 Hz)**, solver/sleep no padrão de cada engine.

Pergunta central: os resultados confirmam o que a arquitetura do Grau A previu?

02

Expectativas & Configuração

O que a teoria prevê · como montamos um teste justo em cada engine

O que esperar de cada motor

Antes de medir, registramos o que a **arquitetura levantada no Grau A** prevê. Na Análise Crítica confrontamos **previsão × dado**.

	Unity · PhysX	Godot · Jolt	Unreal · Chaos
Torre (estabilidade)	Gauss-Seidel iterativo + warm starting → jitter residual em pilhas altas	Impulse-based; multi-thread ajuda pouco (cena quase estática)	XPBD + Shock Propagation — mecanismo feito p/ este cenário
Chuva (desempenho)	Broadphase é o gargalo >5k; FPS cai junto com o Step Time	Job system nativo → melhor escalabilidade esperada em 5k/10k	Async Physics Tick desacopla FPS; LWC eleva o custo base

Guardem estas 3 previsões: Shock Propagation ajuda a Torre? · Jolt escala melhor? · Chaos é mais caro? — a prática responde adiante.

Mesma cena nas 3 engines — só o motor muda

Comparação justa exige travar tudo que não é o motor de física. Estes valores são idênticos nas três.

Parâmetro	Valor	Parâmetro	Valor
Fixed timestep	0,02 s (50 Hz)	Gravidade	≈ (0, -9,81, 0)
Formas	primitivas (cubo / esfera)	Massa / atrito	1 kg / 0,5
Build	Release · V-Sync desligado (todas). Render livre em Unity/Godot; Unreal trava a 50 Hz = mecanismo do passo 0,02 s (ver 2c)	Execuções	15 por configuração
Torre	100 cubos + arena fechada 60×60×150 m	Chuva	1k / 5k / 10k esferas, caixa fechada
Cena	100% gerada em código (C# / C++ / GDScript) — posição exata, reproduzível, zero configuração manual de Inspector		

Duas decisões de método nossas (iguais nas 3): arena fechada na Torre (contém cubos ejetados → métrica limpa) e janela por **tempo de simulação** na Chuva (não wall-clock — ver Bastidores).

Como configuramos cada engine

O que muda entre elas é o **motor no seu padrão de fábrica** — é exatamente isso que comparamos. Nada de solver tunado na coluna principal.

	Unity	Godot	Unreal
Versão	6000.5 (U6 LTS)	4.7	5.8
Motor físico	PhysX (GameObject)	Jolt	Chaos
Solver (default)	PGS · 6+1 iter	impulse (Gauss-Seidel)	XPBD + Shock Prop. · 8+2 iter
Sleep threshold	0,005 m/s	0,03 m/s	5 frames
Física ↔ render	desacoplada	desacoplada	acoplada — trava o frame a 50 Hz p/ garantir o passo fixo 0,02 s (não é um limite de desempenho)
Step time medido via	Stopwatch(Physics.Simulate)	TIME_PHYSICS_PROCESS	TG_Pre→PostPhysics

Ponto p/ o professor: deixar solver e sleep no **padrão de cada engine** é **exigência do spec (§2.1: "não modificar — é parte do que se compara")**, não descuido. Como cada engine dorme a um limiar diferente (0,005 · 0,03 · 5 frames), o "tempo até dormir" NÃO é comparável 1-a-1 → a Torre é lida por **colapso** (geométrico, independe do sleep).

Máquina de teste

As 3 engines rodaram na MESMA máquina — então qualquer diferença nos números vem do **motor de física**, não do hardware. (Ex.: a diferença de ~11× do Chaos vs PhysX em 10k é no mesmo PC.)

Componente	Especificação
CPU	Intel Core i7-12650H (12ª gen, 2,30 GHz) — 10 núcleos (6P + 4E), 16 threads
GPU	NVIDIA GeForce RTX 4050 Laptop (6 GB) + Intel UHD (integrada)
Memória (RAM)	32 GB
Armazenamento	477 GB (347 GB em uso)
Sistema	Windows 64-bit (x64)

Por que importa: num notebook, boost-clock, throttling térmico e o **escalonamento entre P-cores e E-cores** (6P + 4E) variam entre execuções — parte do ruído que o filtro $M\pm\sigma$ trata, e a hipótese mais provável do **FPS bimodal** visto no 10k.

Rigor estatístico — 4 passos por configuração

Uma única execução não é confiável (ruído do SO, agendamento de threads). Repetimos e filtramos.

- **1** · Rodar **15 execuções isoladas** por variação por engine.
- **2** · Calcular **média (M)** e **desvio padrão (σ)** (populacional, $\div N$).
- **3** · **Descartar** execuções fora da faixa $[M - \sigma, M + \sigma]$.
- **4** · **Média Final** = média apenas dos valores dentro da faixa.

Ressalva honesta: em distribuições **bimodais** ou com **timeout**, a Média Final perde sentido → ali usamos **mediana + grupos** (detalhe nos Bastidores).

03

Cenário 1 — A Torre

Estabilidade, jitter e colapso

Uma pilha alta é um teste brutal do solver: o contato da base precisa sustentar o peso de ~100 cubos. Com poucas iterações, o solver **sub-resolve** essa força a cada passo → a coluna afunda (pancaking). Medimos o **tempo até repouso total (sleep)**, se há **jitter** excessivo, e se a física **colapsa**.

Ligação com o Grau A: é exatamente aqui que PGS vs TGS vs XPBD e o número de iterações importam.

O que medimos — e o que de fato compara

Nem toda métrica é **comparável entre engines**. Separamos a de comparação das diagnósticas (glossário completo em `results/METRICS-e-exclusoes.md`).

Métrica	O que mede	Comparável entre engines?
Colapso <code>kept%</code>	Altura final do topo ÷ altura se intacta. Puramente geométrica.	✓ SIM — independe do sleep. Métrica primária.
Tempo até sleep	Tempo até todos os corpos dormirem; reflete a convergência do solver.	✗ não — limiar de sleep difere (0,005 · 0,03 · 5 frames).
Vel. máx <code>max_v</code>	Pico de velocidade na execução. Distingue pancaking (~gravidade) de explosão.	diagnóstico
Timeout (60 s)	Pilha nunca assenta — resultado válido , não outlier.	diagnóstico

Veredito de colapso: ESTÁVEL >90% · PARCIAL 25–90% · COLAPSOU <25%. Lemos a Torre por `kept%` porque é o único critério que mede a **mesma coisa** nas 3 engines.

N = 100 cubos — o que aconteceu

Métrica primária = **colapso** (kept% = altura final ÷ altura intacta) — é geométrica e **independe do limiar de sleep**, então é comparável entre engines.

	Unity · PhysX (PGS)	Godot · Jolt	Unreal · Chaos
Colapsa? (kept%)	💥 sim — 2%	💥 sim — 2%	💥 sim — 2%
Execuções	15/15 colapsam	15/15 colapsam	15/15 colapsam
Velocidade máx.	~40 m/s	~36 m/s	~43 m/s

Achado: as **três colapsam igual** (kept ~2%, pancaking — top_y cai monotônico). Nenhum sustenta 100 cubos com os defaults — o Chaos (Unreal), agora **medido** (15/15), confirma o que a varredura já indicava.

Limiar de colapso — e o "degrau" do Chaos

Varredura de N crescente — **15 execuções por N**, mais o N=100. Todos colapsam cedo, mas o **Chaos aguenta um degrau a mais** em N=15 (kept% = quanto da pilha ficou de pé).

N	Unity · PGS	Godot · Jolt	Unreal · Chaos
10	✓ estável · 99%	✓ estável · 100%	✓ estável · 100%
15	✱ 6%	✱ 22%	⚠ parcial · 38%
20	✱ 9%	✱ 10%	✱ 8%
100	✱ 2%	✱ 2%	✱ 2%

Consistente com o Grau A: só o Chaos tem **Shock Propagation** (default) — segura $\sim\frac{1}{3}$ da pilha em N=15 onde os outros achatam. **Mas efeito limitado:** em N=20 todos colapsam. Nenhum sustenta pilha alta com defaults. (TGS opt-in do Unity sobe o limiar 1 degrau; iterações 6→20 não movem — é o algoritmo, não a contagem.)

O colapso da Torre em ação



Vídeo: o colapso da Torre (pancaking do solver)

▶ **abrir no Drive**

04

Cenário 2 — A Chuva

Estresse de colisão e desempenho

Milhares de corpos caindo juntos saturam o pipeline: a **broadphase** explode em número de pares, a **narrowphase** testa cada contato, o **solver** resolve tudo por passo. Se o passo não termina em 20 ms (50 Hz), a simulação entra em espiral da morte. Medimos **Physics Step Time** (primário) e **FPS**.

Ligação com o Grau A: aqui o threading e a escalabilidade do solver (Jolt job system vs Chaos islands) aparecem nos números.

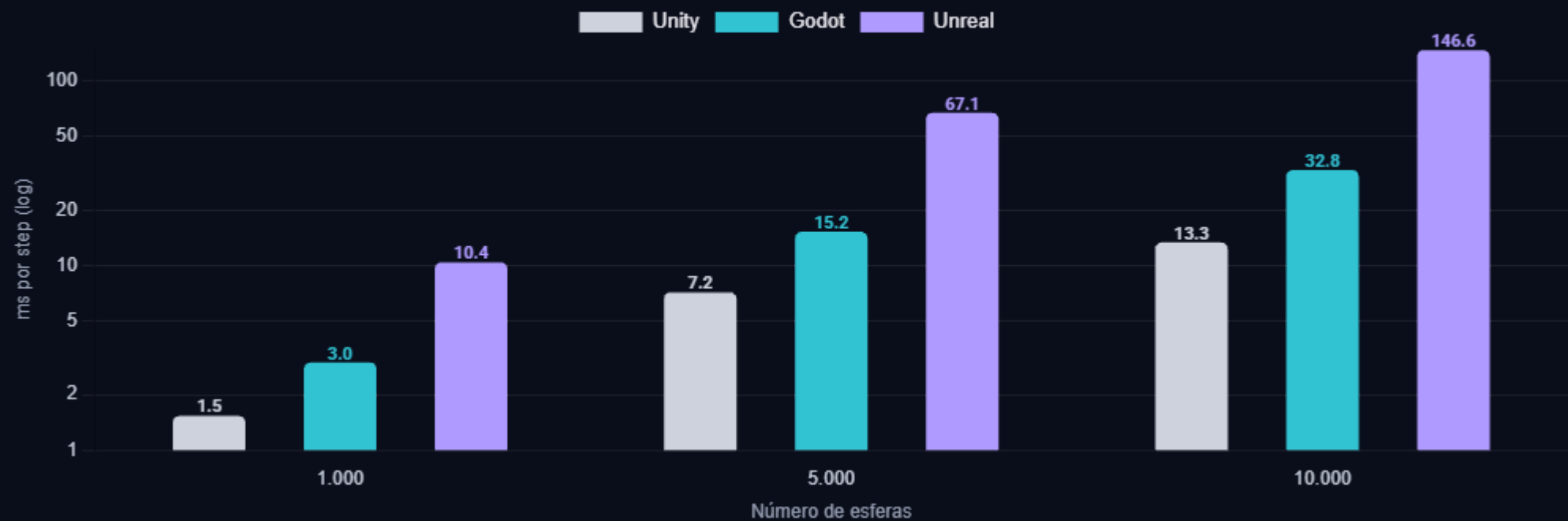
O que medimos — e como as métricas se ligam

Duas métricas do enunciado. Uma **compara entre engines**, a outra só vale **por engine** — e a razão é o acoplamento render↔física.

Métrica	O que mede	Comparável entre engines?
Physics Step Time	Custo de CPU para avançar 1 passo de 0,02 s com N corpos.	✓ SIM — mesma grandeza física nas 3. Métrica primária.
Step time máx.	Pior passo isolado da janela (o pico quando a pilha se forma).	diagnóstico de pico
FPS médio	Ritmo do laço de render (não da física).	✗ não — render acoplado (Unreal) vs livre (Unity/Godot).
amostras	Nº de passos de física na janela (alvo 500 = 10 s simulados).	auditoria da janela

Como se ligam: com física **acoplada** (Unreal) FPS ≈ 1000/step (teto 50); **desacoplada** (Unity/Godot) o FPS é velocidade de GPU. Por isso o **Step Time compara** e o FPS não — e amostras=500 confirma que as 3 mediram o mesmo trecho simulado.

Physics Step Time — quanto custa cada passo



5k → Unity 7,1 · Jolt 15,2 · Chaos 67,1 ms

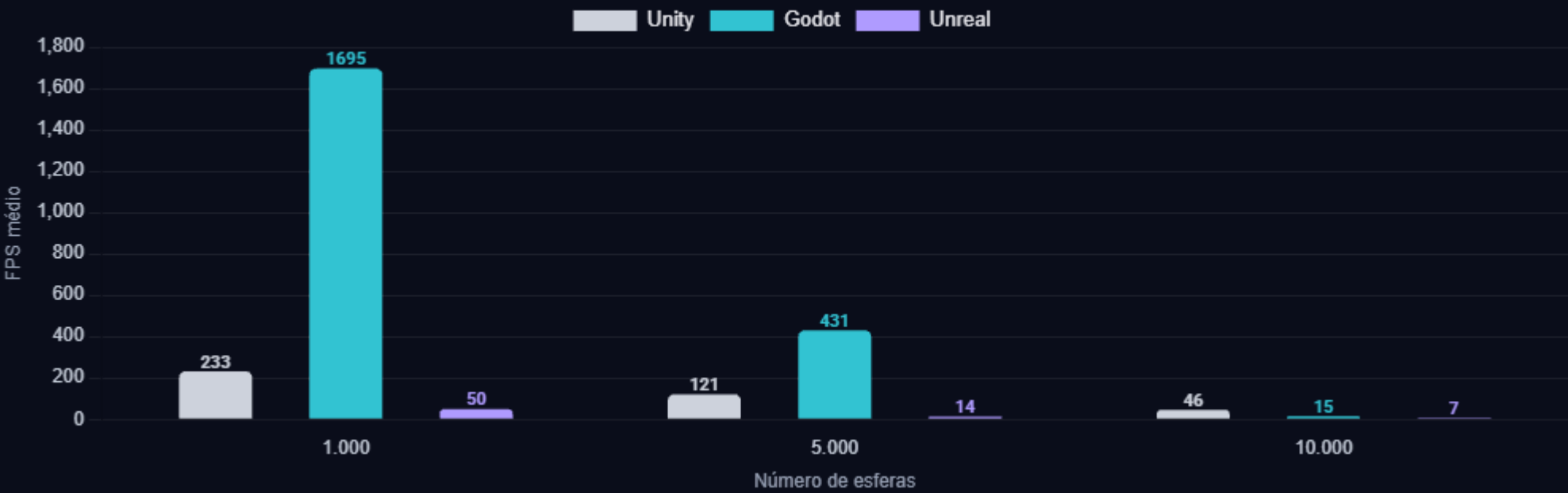
10k → Unity 13,3 · Jolt 32,8 · Chaos 146,6 ms

Classificação: Unity < Godot < Unreal

Custo relativo (× vs PhysX, o mais barato) em 10k: PhysX 1× · Jolt 2,5× · Chaos 11×. Eixo Y em **escala log** (cada divisória = 10×) — é o que deixa a barra do PhysX (13,3 ms) legível ao lado do Chaos (146,6 ms).

FPS sob estresse — com uma ressalva importante

Primeiro, o que É igual: o passo de física é 0,02 s (50 Hz) nas três — a exigência do spec está cumprida. O que **não** é comparável 1-a-1 é o **FPS**, e a causa é o **acoplamento render↔física**, não o timestep. Na **Unreal** a física roda **acoplada** (1 passo por frame) → o FPS reflete o custo da física (teto ~50). Em **Unity/Godot** o render é **desacoplado e livre** → ali "FPS" é velocidade de render/GPU, não custo de física. Por isso o FPS é lido **por engine** (tendência) e o **Physics Step Time é a métrica primária** que compara entre engines.



Como cada motor escala

Unity · PhysX

Mais barato em toda escala: 1,5 / 7,1 / 13,3 ms. 1k→10k = **8,7×** **baseline** (**1×**). Broadphase SAP eficiente; não precisou tunar nada.

Godot · Jolt

3,0 / 15,2 / 32,8 ms. 1k→10k = **11,0×** · **≈2,5×** o PhysX em 10k. Precisou **tunar buffers do Jolt** em 10k (senão trava).

Unreal · Chaos

10,4 / 67,1 / 146,6 ms — **o mais caro**. 1k→10k = **14,1×** · **≈11×** o PhysX em 10k. Custo alto (XPBD + LWC + acoplamento + editor).

Ressalva: o escalonamento 5k→10k é **sublinear nas três** (Unity 1,9× · Jolt 2,2× · Chaos 2,2×) — logo é efeito da **janela de medição** (captura a pilha assentada), **não** uma vantagem específica do Jolt.

Os valores medidos (n=15)

Physics Step Time — Média Final $\pm \sigma$ (dispersão do lote de 15 execuções). É a tabela por trás do gráfico.

Esferas	Unity · PhysX	Godot · Jolt	Unreal · Chaos
1.000	1,53 $\pm$ 0,02 ms	2,98 $\pm$ 0,23 ms	10,39 $\pm$ 0,13 ms
5.000	7,15 $\pm$ 0,07 ms	15,22 $\pm$ 1,08 ms	67,08 $\pm$ 2,39 ms
10.000	13,32 $\pm$ 0,48 ms	32,83 $\pm$ 2,00 ms	146,55 $\pm$ 8,01 ms
FPS médio · 1k / 5k / 10k	233 / 121 / 46	1695 / 431 / 15*	50 / 14 / 7

Custo relativo em 10k (× vs PhysX): Jolt $\approx$ **2,5×** · Chaos $\approx$ **11×**. *FPS Godot 10k é bimodal (mostramos a mediana); o FPS não compara entre engines — lê-se por engine.

CENÁRIO 2 · VÍDEO

10.000 corpos sob estresse



Vídeo: Chuva 10k — onde a física começa a engasgar

▶ **abrir no Drive**

05

Análise Crítica

Estabilidade — quem segurou a pilha?

Q1 · Qual suportou mais corpos sem colapso? Nenhum sustenta N=100. Pelo limiar de colapso, o **Chaos (Unreal)** aguenta um degrau a mais — parcial em N=15 (Shock Propagation) — enquanto Jolt e PGS colapsam já em N=15. Em N=20 todos colapsam.

Q2 · Qual entregou a pilha mais estável? Pelo critério comparável (**colapso / kept%**), **Chaos** — único a segurar $\sim \frac{1}{3}$ da pilha em N=15. O tempo-até-sleep NÃO serve de critério: cada engine dorme a um limiar diferente (0,005 · 0,03 · 5 frames).

Leitura: a "pilha estável" não existe de fábrica em nenhuma — o design de jogos evita pilhas rígidas altas justamente por isso.

Desempenho & o veredito sobre o Grau A

Q3 · Chaos vs PhysX vs Jolt? Na Chuva (Step Time), a classificação é **Unity/PhysX < Godot/Jolt < Unreal/Chaos**: PhysX é o mais barato (13,3 ms em 10k), ~**11×** mais rápido que o Chaos (146,6 ms). Chaos paga XPBD + LWC + acoplamento.

Q4 · Os resultados confirmam o Grau A? Em parte. Confirmou: o Chaos é o mais caro (LWC + XPBD) e o **Shock Propagation** segura 1 degrau a mais na Torre (N=15). **Inverteu:** esperávamos o **Jolt** (Godot) como o mais escalável na Chuva — mas o mais barato em toda escala foi o **PhysX** (Unity); o Jolt ficou no meio ($\approx 2,5\times$ o PhysX em 10k). **Ressalva:** o FPS não é comparável 1-a-1 (acoplamento render $\leftrightarrow$ física). **Ainda corrigimos 2 suposições de método:** (1) o solver padrão do Unity é **PGS**, não TGS; (2) a "diferença de 11 $\times$ " é **custo bruto da configuração** (editor + física acoplada + sincronização de 10k atores), não só do solver.

O benchmark tornou a teoria mensurável — e corrigiu duas suposições do Grau A (PGS $\neq$ TGS; a leitura da diferença do Chaos).

06

Bastidores

Acertos, erros, ajustes e experimentos — o que o número final não mostra

Erros que cometemos — e corrigimos

Problema encontrado	Correção	Impacto
Janela em wall-clock: em 10k, 10 s reais = só ~2 s simulados (queda livre, zero colisão)	Janela por tempo de simulação (500 passos), idêntica nas 3	Chaos 10k saltou ~86 → 147 ms (mede a pilha densa de fato)
Unity: <code>ProfilerRecorder</code> retornava 0 no step time	<code>Stopwatch</code> em volta de <code>Physics.Simulate()</code>	1 execução inicial descartada; métrica nunca mais veio 0
Assumimos TGS no Unity (herança do Grau A)	Config real = PGS (<code>m_SolverType:0</code>)	Grau A corrigido; previsão de jitter revista
Jolt: buffer de contatos estourou em 10k (travamento na 2ª execução)	Tunar buffers do Jolt + reset in-place entre execuções	10k passou a rodar sem travamento

Rigor é isto: cada número final só vale porque um erro de método foi pego antes de virar resultado.

Desvios do spec — e por que são justos

Ajustamos alguns parâmetros do enunciado original. **Regra de ouro:** cada mudança foi aplicada **idêntica nas 3 engines** → a comparação cross-engine continua justa. Os que mais pesam:

Parâmetro	Spec original	O que rodou	Por quê
Container (Chuva)	funil (pirâmide)	caixa fechada 35x80x35 m	reprodutível + contém os corpos → métrica limpa
Sleep threshold	0,05 m/s	default de cada engine (0,005 · 0,03 · 5 frames)	§2.1 proíbe mexer no sleep → a Torre é lida por colapso
Janela (Chuva)	30 s wall-clock	500 passos de simulação (~10 s sim)	wall-clock media trechos físicos diferentes por engine (bug)
Menores	forma box→ esfera · atrito 0,6/0,8→ 0,5 · restituição Chuva 0→0,3 · spawn aleatório→ grade 3D · arena fechada na Torre		impacto baixo; iguais nas 3

Nenhum desvio favorece uma engine sobre outra — todas rodaram a MESMA cena. O único "com peso" (funil→caixa) foi p/ ter uma métrica reprodutível. Tudo registrado em SCENE_SPEC.md → "Desvios do spec".

Experimentos deliberados

"Otimizar" o Chaos

Cortamos iterações 8→4 + CVars.
Resultado: **+19% mais lento** em 10k.
Aprendizado: o gargalo é o pipeline de **colisão**, não as iterações.

PGS vs TGS vs +iter

Iterações 6→20 **não movem** o limiar de colapso. TGS (opt-in) sobe **1 degrau** (N=15). É o algoritmo, não a contagem.

Arena fechada

Decisão de método: paredes + teto p/ conter cubos ejetados → métrica de colapso válida em todas as engines, sem timeouts falsos.

A "otimização que piorou" é o achado mais honesto do trabalho — e o **porquê** vem no próximo slide.

A "otimização" que piorou o Chaos — por quê?

O experimento: cortamos as iterações do solver (8→4), desligamos CCD, adiamos a narrowphase e fatiamos em mais workers. **Resultado:** em 10k, **146,6 → 173,8 ms (+19%)** — e o FPS caiu junto. As hipóteses:

- **1 · O gargalo não são as iterações do solver.** Numa chuva densa (10k empilhados) o custo dominante é o **pipeline de colisão** (broad + narrowphase, contagem de contatos) sobre milhares de pares. Cortar iterações raspa só a parte barata.
- **2 · Menos iterações = pilha menos estável = MAIS trabalho.** Com 4 iterações a pilha assenta pior; mais corpos ficam **acordados e tremendo** → mais contatos ativos por passo → o solver de colisão trabalha mais. A economia é **engolida** pelo contato extra.
- **3 · Mais workers foi contraproducente.** Uma chuva empilhada é **uma ilha de contato gigante**; fatiá-la em mais grupos só somou overhead de agendamento/sync, sem paralelismo real (não se paraleliza uma ilha única).
- **4 · CCD off / defer narrowphase não moveram o ponteiro** — as esferas não usavam CCD, e adiar a narrowphase só reorganiza o mesmo custo. (Bônus ruim: perdemos o determinismo — σ subiu de ≈ 0 para 7,5 ms.)

A lição: os ~150 ms do Chaos em 10k são **estruturais** (arquitetura de colisão + LWC + 10k atores), não um default conservador que se destrava por CVar. Uma otimização que **falha**, bem explicada, ensina tanto quanto uma que funciona.

Quantas vezes rodamos — e o que descartamos

Cada configuração roda 15× (≈315 execuções no total). O filtro do enunciado joga fora o que cai **fora da faixa** $[M-\sigma, M+\sigma]$; a **Média Final** é a média só do que sobra.

Configuração	Execuções descartadas · Unity / Godot / Unreal
Chuva 1.000	5 / 5 / 3
Chuva 5.000	1 / 5 / 7
Chuva 10.000	1 / 5 / 4
Torre N=100	4 / 4 / 1

Descartar ~1/3 é ESPERADO — não é dado ruim. A faixa $\pm 1\sigma$ contém ~68% dos valores, então ~1/3 cai fora **por definição**. (O Unreal descarta só 1 na Torre porque o **Chaos é quase determinístico**: as 15 runs saem quase idênticas, $\sigma \approx 0$.)

Dois datasets — n=10 e n=15 (e por que batem)

Rodamos **dois lotes independentes** — um piloto de **10** e o final de **15** execuções. Se o número mal se move ao subir de 10→15, ele é **confiável**. Vale para os **dois cenários**:

Chuva · Step Time 10k (ms)	Unity	Godot	Unreal
n=10 → n=15	13,1 → 13,3	32,9 → 32,8	149,8 → 146,6
Δ	+1,7%	-0,1%	-2,2%

Torre · colapso kept%	Unity	Godot	Unreal
N=15 · n=10 → n=15	8% → 6%	14% → 22%	38% → 38%
Veredito (não muda)	COLAPSOU	COLAPSOU	PARCIAL

Nada mudou de 10→15: toda a Chuva se moveu **< 7%** (pior caso Unity 1k, +6,4%) e os **vereditos da Torre são idênticos**. → dataset **reprodutível**; o n=15 é o canônico (o n=10 fica só para esta comparação).

07

Considerações Finais

Síntese e limites

- **Estabilidade:** nenhum motor sustenta pilhas rígidas altas com defaults — é arquitetura do solver, não iterações. O Chaos aguenta 1 degrau a mais (Shock Propagation).
- **Desempenho:** na Chuva, **Unity/PhysX < Godot/Jolt < Unreal/Chaos** (Step Time). PhysX é ~11× mais barato que o Chaos em 10k; o sublinear 5k→10k aparece nas três (efeito de janela).
- **Limitações honestas:** a "diferença de 11×" do Chaos inclui editor + acoplamento + sincronização de 10k atores (não é só o solver); FPS não comparável 1-a-1 (Unreal capado a 50 Hz); FPS 10k bimodal (lido por mediana + grupos).

Fechamento: a arquitetura do Grau A previu bem o comportamento — e o benchmark o tornou mensurável, **corrigindo pelo caminho** duas suposições (PGS≠TGS; a leitura da diferença do Chaos).

Fontes & código

- **Repositório:** código das 3 engines + CSVs brutos + [RESULTS.md](#), [COMPARISON-n10-vs-n15.md](#), [METRICS-e-exclusoes.md](#) [link GitHub — a inserir](#)
- **Metodologia:** spec do assignment-b · [howto-statistics.md](#) (método $M \pm \sigma$).
- **Grau A:** levantamento arquitetural que fundamenta as hipóteses (PhysX / Chaos / Jolt).
- Rouwé, J. — **Architecting Jolt Physics for 'Horizon Forbidden West'**, GDC 2022.
- Macklin, M. et al. — **Small Steps in Physics Simulation**, SIGGRAPH/SCA 2019 (XPBD com substeps).

Obrigado

Física para Jogos Digitais · Benchmark de Physics Engines

Docente: Prof.^a Rossana Baptista Queiroz

Acadêmico: Matheus Dubin da Silveira

